

Fesztivál

Nayra egy fesztiválon egy játékot játszik, melynek fődíja egy utazás Laguna Coloradába. A játék lényege, hogy zsetonokkal kuponokat vásárolsz. Kupon vásárlásával további zsetonokat szerezhetsz. A cél az, hogy minél több kupont szerezz.

A játékot Nayra A darab zsetonnal kezdi. N kupon van, amelyek számozása 0 -tól $N - 1$ -ig terjed. Nayrának $P[i]$ zsetont kell fizetnie, hogy megvásárolhassa az i -edik kupont (és a vásárlás előtt legalább $P[i]$ zsetonnal kell rendelkeznie). Minden kupont legfeljebb egyszer vehet meg.

Ezenkívül minden i -re ($0 \leq i < N$) az i -edik kuponnak van egy **típusa**, amelyet $T[i]$ jelöl, és **1, 2, 3 vagy 4** lehet. Miután Nayra megvásárolta az i -edik kupont, a fennmaradó zsetonjainak száma megszorozódik $T[i]$ -vel. Formálisan, ha a játék egy bizonyos pontján X zsetonja van, és megveszi az i -edik kupont (amihez $X \geq P[i]$ szükséges), akkor a vásárlás után $(X - P[i]) \cdot T[i]$ zsetonja lesz.

A feladatod annak meghatározása, hogy Nayra melyik kuponokat és milyen sorrendben vásárolja meg, hogy a végén minél több **kuponja** legyen. Ha a vásárlásoknak több olyan sorrendje is van, amellyel ez elérhető, bármelyiket megadhatod.

Megvalósítás

A következő függvényt kell megvalósítanod:

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : Nayra kezdeti zsetonjainak száma.
- P : egy N elemű tömb, amely a kuponok árait tartalmazza.
- T : egy N elemű tömb, amely a kuponok típusait tartalmazza.
- Ez a függvény minden egyes tesztesetnél pontosan egyszer hívódik meg.

A függvénynek egy R tömböt kell visszaadnia, amely Nayra vásárlásait a következőképpen adja meg:

- Az R tömb hossza legyen a maximálisan megvásárolható kuponok száma.
- A tömb elemei a megvásárolni kívánt kuponok sorszámai időrendi sorrendben. Vagyis először az $R[0]$ -adik kupont vásárolja meg, majd az $R[1]$ -edik kupont, és így tovább.
- Az R minden elemének különbözőnek kell lennie.

Ha nem lehet egy kupont sem vásárolni, akkor az R tömbnek üresnek kell lennie.

Korlátok

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ minden i -re, ahol $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ minden i -re, ahol $0 \leq i < N$.

Részfeladatok

Részfeladat	Pontszám	További megszorítások
1	5	$T[i] = 1$ minden i -re, ahol $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ minden i -re, ahol $0 \leq i < N$.
3	12	$T[i] \leq 2$ minden i -re, ahol $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra meg tudja venni mind az N kupont (valamilyen sorrendben).
6	16	$(A - P[i]) \cdot T[i] < A$ minden i -re, ahol $0 \leq i < N$.
7	18	Nincsenek további megszorítások.

Példák

1. példa

Tekintsük a következő hívást:

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra kezdetben $A = 13$ zsetonnal rendelkezik. 3 kupont tud venni az alábbi sorrendben:

Megvásárolt kupon	Kupon ára	Kupon típusa	A vásárlás utáni zsetonok száma
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Ebben a példában Nayrának nincs lehetősége 3 kuponnál többet vásárolnia, és a fent leírt vásárlási sorrend az egyetlen módja annak, hogy 3 kupont megvásároljon. Ezért a függvénynek a $[2, 3, 0]$ értékkel kell visszatérnie.

2. példa

Tekintsük a következő hívást:

```
max_coupons(9, [6, 5], [2, 3])
```

Ebben a példában Nayra mindkét kupont meg tudja vásárolni, tetszőleges sorrendben. Ezért a függvények vagy a $[0, 1]$, vagy az $[1, 0]$ értékkel kell visszatérnie.

3. példa

Tekintsük a következő hívást:

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Ebben a példában Nayrának egy zsetonja van, ami nem elég ahhoz, hogy kuponokat vásároljon. Ezért a függvénynek $[]$ (üres tömb) értékkel kell visszatérnie.

Mintaértékelő

Bemeneti formátum:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Kimeneti formátum:

```
S
R[0] R[1] ... R[S-1]
```

Itt S a `max_coupons` függvény által visszaadott R tömb hossza.